EXPERIMENT NO: 5

Student Name: Mahak Singh UID: 23BCS12883

Branch: BE-CSE Sec on/Group: KRG-2A

Semester: 6th Date of Performance: 23/02/2026

Subject Name: System Design Subject Code: 23CSH-314

Aim

To design a scalable real- me messenger applica on similar to WhatsApp or Facebook Messenger that supports one-to-one and group messaging with high availability, low latency, and reliable message delivery.

Objec ves

- Understand the architecture of a real- me messaging system.
- Iden fy func onal requirements such as authen ca on, chat, and media sharing.
- Iden fy non-func onal requirements including availability, reliability, and latency constraints.
- Analyze CAP theorem trade-offs for messaging systems.
- Design REST and WebSocket APIs for chat communica on.

Procedure

1. Studied real-world messaging systems like WhatsApp and Facebook Messenger.
2. Iden fied core en es such as Users, Groups, and Messages.
3. Analyzed real- me communica on using WebSockets.
4. Collected func onal and non-func onal requirements.
5. Designed APIs for user authen ca on and messaging.
6. Evaluated storage and scalability requirements for large-scale systems.
7. Analyzed availability and eventual consistency trade-offs.

Func onal Requirements

1. Client should be able to register and login into the system.
2. System should support one-to-one messaging.
3. System should support group messaging.
4. System should support sending textual as well as media messages.
5. System should preserve complete message history.
6. Client should be able to see delivered and read receipts.

7. Messages should be received in real- me.

Core En es of the System

- Users
- Groups
- Messages

API Design

1. User Registra on API: POST /user/register
2. User Login API: POST /user/login

One-to-One Messaging

A. Send Message (WebSocket): WS /messages/send
B. Get Chat List: GET /chat/{user_id} (Pagina on Supported)
C. Get Messages of Specific Chat: GET /messages/{user_id}/{receiver_id}

Group Messaging

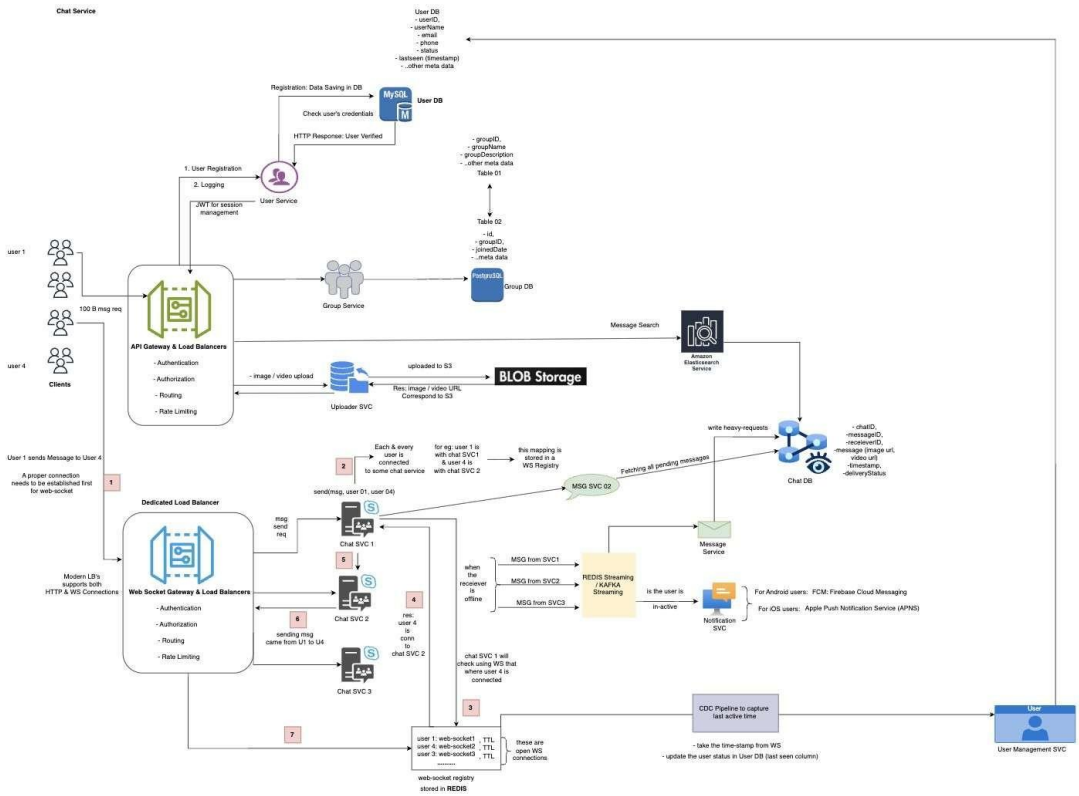
A. Create Group: POST /groups/create
B. Send Group Message (WebSocket): WS /messages/send
C. Get Group Chats: GET /groups/{group_id} (Pagina on Supported)
D. Add Member: POST /groups/{group_id}/add
E. Remove Member: DELETE /groups/{group_id}/remove

Non-Func onal Requirements

1. System should handle extremely large scale traffic (millions to billions of messages per day) requiring approximately 100 TB storage.
2. High Availability should be priori zed under CAP theorem.
3. System should follow Eventual Consistency model.
4. End-to-end latency should be approximately 200–300 ms.
5. System should be highly reliable with no message loss or packet drop.
6. Horizontally scalable distributed architecture.

High Level Design (HLD)

The system consists of Client Applica ons (Mobile/Web), API Gateway, Authen ca on Service, Messaging Service (WebSocket Servers), Group Service, Media Service, Message Queue, Distributed Databases, Cache Layer (Redis), and Object Storage for media files. Messages are delivered through
WebSocket servers for real- me communica on and stored in distributed databases for persistence.



Learning Outcomes

- Designed a scalable real- me messaging system.
- Iden fied func onal and non-func onal requirements.
- Designed REST and WebSocket APIs.
- Understood high availability and eventual consistency trade-offs in messaging systems.